Техническая документация Felex v1.0.1

Аннотация

Felex v1.0.1 — локальная программная система для расчёта, анализа и оптимизации рационов кормления сельскохозяйственных животных. Система объединяет курируемую библиотеку кормов, модуль расчёта питательности, модуль экономического анализа, многостадийный модуль оптимизации рациона, инструменты контроля цен, экспорт документов и AI-ассистент, работающий через локальный сервер Ollama. Настоящая документация описывает архитектуру, модель данных, вычислительные процедуры, контур валидации и ограничения системы в форме, пригодной для включения в техническое приложение или научную публикацию.

1. Назначение системы

Система предназначена для решения следующих задач:

- хранение и просмотр профилей кормов;
- формирование рационов по видам животных и производственным направлениям;
- вычисление фактического поступления энергии, протеина, клетчатки, минералов и витаминов;
- сравнение фактического состава с нормативным эталоном;
- оценка стоимости рациона;
- оптимизация рациона по критерию стоимости, приоритетного нутриентного баланса или балансировки с жёсткой фиксацией структуры;
- документирование результата в форматах PDF, CSV и XLSX.

2. Архитектура

2.1. Общая схема

Архитектура Felex включает четыре основные подсистемы:

Подсистема	Назначение
Frontend	пользовательский интерфейс на React/Vite
Embedded API	HTTP API на Rust/Axum
Desktop shell	Tauri-оболочка, команды экспорта и открытия внешних ссылок
Local AI	локальный LLM backend через Ollama

Frontend работает как основная рабочая среда оператора. В desktop-режиме он взаимодействует с локальным HTTP API и Tauri-командами.

2.2. Стек реализации

- **Frontend**: React, TypeScript, Zustand, i18next, React Markdown.
- **Backend**: Rust, Axum, rusqlite.
- **Desktop layer**: Tauri 2.
- **Export**: `printpdf`, `rust_xlsxwriter`, CSV-генерация.
- **LLM**: Ollama API (`/api/chat`).

3. Модель данных

3.1. Сущность `Feed`

Объект корма содержит:

- идентификаторы и служебные поля источника;
- наименование на русском и английском языках;
- категорию и подкатегорию;
- нутриентные показатели;
- экономические показатели;
- служебные флаги верификации и пользовательского происхождения.

Ключевые группы нутриентов:

- сухое вещество;
- обменная энергия;
- сырой и переваримый протеин;
- клетчатка;
- кальций, фосфор и другие макроэлементы;
- микроэлементы;
- витамины.

3.2. Сущность `FeedPrice`

Текущая цена корма хранится отдельно от основного feed-профиля. Запись цены включает:

- `feed_id`;
- регион;
- цену за тонну;
- дату цены;
- тип источника;
- текстовые заметки;
- историю для последующего аудита.

Такое разделение позволяет независимо обновлять цены без модификации основного нутриентного профиля.

3.3. Сущность рациона

Рацион хранит:

- идентификатор животной группы;
- свойства животных;
- число голов;
- список кормов;
- количество каждого корма на голову в сутки;
- признаки фиксации отдельных ингредиентов;

- активный набор норм;
- пользовательские корректировки норм.

4. Нормативная база

Система использует встроенные пресеты норм для основных производственных групп. Для каждого показателя может быть задан:

- минимум;
- целевое значение;
- максимум.

Расчётный модуль сравнивает фактическое значение с интервалом и формирует статус показателя. Пользователь может:

- использовать встроенный пресет;
- выбрать другой нормативный пресет как эталон;
- вручную изменить отдельные границы и целевые значения.

5. Расчёт питательности

5.1. Основной принцип

Для каждого ингредиента определяется вклад по каждому нутриенту как произведение количества корма на соответствующую концентрацию показателя. Далее вклады суммируются по всем кормам.

Формально, для нутриента `j` в рационе из `n` ингредиентов:

`N_j = Σ(i=1..n) q_i * c_ij`

где:

- `q_i` — количество `i`-го корма на голову в сутки;
- `c_ij` — концентрация нутриента `j` в `i`-м корме;
- `N_j` — суммарное поступление нутриента `j`.

5.2. Масштабирование на группу

Если число голов равно `H`, то групповой расход и групповая стоимость вычисляются как:

- `Q_group = H * Q_head`
- `C_group = H * C_head`

Это масштабирование применяется во всех пользовательских панелях, строке состояния и экспортируемых отчётах.

6. Экономический модуль

Экономический модуль использует цену корма за тонну. Цена на килограмм вычисляется делением на 1000. Суточная стоимость ингредиента на голову:

`Cost_i = q_i * (Price_i / 1000)`

Общая стоимость рациона на голову:

`Cost_head = Σ(i=1..n) Cost_i`

Стоимость на группу:

`Cost_group = H * Cost_head`

Дополнительно система готовит производные показатели для отчёта:

- общая масса рациона на голову;
- общая масса рациона на группу;
- суточная стоимость на голову;
- суточная стоимость на группу;
- месячная и годовая оценка для группы.

7. Модуль цен

7.1. Источники данных

Модуль цен работает в многослойном режиме:

1. курируемый seed-набор, встроенный в дистрибутив;
2. автоматический веб-сборщик цен;
3. ручной ввод пользователя.

7.2. Нормализация и сопоставление

Web-fetcher приводит текст кормов к канонической форме, используя нормализацию регистра, пробелов и русской графики (`е/ё`). Затем цена сопоставляется с feed-записью по каноническому ключу и по совпадению имён.

7.3. Контроль релевантности ссылок

В версии 1.0 механизм источников был усилен:

- результаты веб-поиска фильтруются по ключевым признакам конкретного корма;
- прямые ссылки показываются только для feed-специфичных источников;
- оценочные benchmark-цены не сопровождаются вводящей в заблуждение прямой ссылкой;
- вместо этого пользователю предлагается feed-специфичный поисковый запрос для валидации.

Это снижает риск неверной интерпретации цены при наличии только ориентировочного рыночного эталона.

8. Модуль оптимизации

8.1. Режимы работы

Бэкенд поддерживает три режима оптимизации:

- `minimize_cost` решает классическую LP-задачу по текущему набору незаблокированных кормов;
- `balance` запускает гибридный многостадийный контур балансировки и используется в интерфейсе по умолчанию;
- `fixed` использует ту же балансировку, но с более узкими пределами изменения количеств, когда рацион должен остаться близким к проекту пользователя.

8.2. Этап 0: стартовый рацион для пустых черновиков

Если рацион пуст и пользователь запускает оптимизацию из интерфейса, бэкенд сначала строит стартовый план из библиотеки кормов.

- корма классифицируются по группам: грубые, сочные, концентраты, белковые, животного происхождения, минеральные, премиксы, витаминные и прочие;
- для активной группы животных подбираются корма из обязательных групп и рассчитываются стартовые количества по шаблону;
- в ответе такой запуск помечается как `auto_populated = true`, а в `applied_strategy` добавляется суффикс `+starter_plan`.

8.3. Этап 1: скрининг и проверка выполнимости

Перед финальной балансировкой текущий набор кормов сравнивается с активным набором норм.

- этап скрининга определяет лимитирующие нутриенты, которые нельзя покрыть текущими ингредиентами;
- из библиотеки ранжируются кандидаты на добавление и возвращаются как `FeedRecommendation` с причиной, категорией, приоритетом и рекомендуемым количеством;
- действительно невыполнимые случаи больше не маскируются как "решение без изменений": оптимизатор возвращает `Infeasible`.

8.4. Этап 2: приоритетно-иерархическая балансировка

Балансировщик больше не штрафует изменение количеств как основную цель. Вместо этого используются ограниченные диапазоны перемещения кормов и последовательные проходы по приоритетам вида животных.

- КРС: уровень 1 `energy_eke`, `crude_protein`, `dig_protein_cattle`; уровень 2 `ndf_pct_dm`, `calcium`, `phosphorus`; уровень 3 `selenium`, `vit_a`, `vit_d3`, `vit_e`;
- свиньи: уровень 1 `energy_oe_pig`, `lysine_sid`, `crude_protein_pct`; уровень 2 `methionine_cystine_sid`, `calcium`, `phosphorus_avail`; уровень 3 `vit_e`;
- птица: уровень 1 `energy_oe_poultry`, `crude_protein_pct`, `lysine_tid_pct`, `methionine_cystine_tid_pct`; уровень 2 `calcium_pct`, `phosphorus_avail_pct`; уровень 3 `vit_e`.

В режиме balance каждый незаблокированный корм может измениться в пределах `max(±60%, ±5 кг)` от текущего количества. В режиме fixed это окно сужено до `max(±25%, ±2,5 кг)`.

8.5. Этап 3: экономический проход и состав ответа

После фиксации нутриентных уровней в допустимых коридорах выполняется дополнительный стоимостной проход, который ищет самое дешёвое решение внутри уже найденной области выполнимости.

Структура `DietSolution` включает:

- обновлённые количества и итоговую питательность;
- суточную стоимость и предупреждения валидатора;
- `recommendations` из этапа скрининга;
- `applied_strategy` с описанием реально выполненного сценария;
- `auto_populated`, если перед оптимизацией был автоматически построен стартовый рацион.

8.6. Интерпретация результата

- `Optimal`: многостадийный решатель нашёл рацион внутри активных ограничений и целевых коридоров.
- `Feasible`: используется для текущего рациона без перестроения, когда шаг оптимизации неприменим.
- `Infeasible`: текущий набор кормов не может удовлетворить нормам в пределах допустимого диапазона изменений. Следующий шаг — добавить рекомендованные корма или запустить авто-заполнение.
- `Unbounded` и `Error`: служебные статусы для ошибок решателя и транспортного слоя.

9. Интеграция локальной LLM

9.1. Роль ассистента

LLM не выполняет основной расчёт нутриентов. Он служит для:

- текстового анализа состояния рациона;
- объяснения дефицитов и избытков;
- генерации рекомендаций по заменам;
- работы с пользовательскими вопросами.

9.2. Встроенный режим рассуждения

Встроенный агент сохраняет режим рассуждения модели включённым. Запросы к Ollama отправляются с `think: true`, а бэкенд обрабатывает раздельные поля рассуждения и финального ответа вместо того, чтобы отключать thinking mode ради обхода ошибок.

- агент дожидается финального пользовательского ответа, даже если модель отдельно выдаёт поток рассуждений;
- ответы, состоящие только из reasoning-части и завершившиеся из-за длины контекста, автоматически запрашиваются повторно;
- изменения размера контекста в интерфейсе теперь реально попадают в бэкенд, поэтому `num_ctx` начинает действовать сразу.

9.3. Устойчивость к ошибкам Ollama

При работе с крупными локальными моделями, например Qwen 3.5 9B, Ollama всё ещё может возвращать

`500 Internal Server Error` или завершать раннер из-за нехватки ресурсов. Поэтому бэкенд использует лестницу повторных попыток.

1. отправить запрос с текущим `num_ctx`;
2. при ответах 5xx последовательно уменьшать размер контекста вплоть до `1024`;
3. если Ollama аварийно завершает `qwen3.5:9b` из-за локальных ресурсных ограничений, повторить тот же запрос на `qwen3.5:4b`;
4. если все попытки неудачны, вернуть диагностическую ошибку вместо пустого ответа.

Такой подход повышает устойчивость AI-подсистемы без отключения режима рассуждения.

10. Экспортный контур

10.1. Поддерживаемые форматы

Система поддерживает экспорт в:

- PDF;
- CSV;
- XLSX.

10.2. Параметры экспорта

Пользователь может задать:

- папку назначения;
- имя файла;
- семейство шрифтов;
- визуальный профиль оформления.

10.3. Содержимое отчёта

Экспортируемый отчёт может включать:

- заголовок и метаданные;
- состав рациона;
- показатели на голову;
- показатели на группу;
- сравнение с нормами;
- экономический блок;
- текстовые примечания.

11. Интерфейс и двуязычность

Система поддерживает русский и английский интерфейс. Для этого используются:

- локализованные словари UI;
- двуязычные описания документов;
- двуязычные Markdown- и PDF-версии встроенной документации.

Такой подход важен для:

- локального использования в русскоязычном хозяйстве;
- подготовки англоязычных приложений, докладов и статей.

12. Надёжность, воспроизводимость и методика бенчмарков

Для воспроизводимости вычислений важны следующие решения:

- фиксированные структуры данных для feed-профилей и норм;
- хранение истории цен;
- явное разделение ручных и автоматически полученных цен;
- экспорт результатов в документированные форматы;
- встроенная техническая документация в тех же терминах, что используются в интерфейсе;
- сохранение снимков бенчмарков рядом с исходным кодом, включая [`docs/benchmarks/2026-03-11-perf_probe.json`](../../docs/benchmarks/2026-03-11-perf_probe.json).

12.1. Методика бенчмарков от 11 марта 2026 года

Замеры выполнены 11 марта 2026 года командой `cargo run --quiet --bin perf_probe` на локальной базе кормов (300 записей).

- микробенчмарки расчёта питательности использовали `25 000`, `15 000` и `8 000` итераций для синтетических рационов на 8, 24 и 64 ингредиента;
- бенчмарки оптимизации использовали `300` итераций для `minimize_cost` и `200` итераций для `balance` и `fixed` в трёх реальных сценариях: `dairy_low_fiber`, `grower_pig_lysine_deficit`, `layer_low_calcium`;
- вспомогательные рабочие этапы использовали `500` итераций для генерации стартового рациона и `500` итераций для feed screening;
- агент тестировался через `/api/v1/agent/chat` по одному запросу на сценарий для `qwen3.5:4b` (`num_ctx = 8192`) и `qwen3.5:9b` (`num_ctx = 4096`).

12.2. Актуальные результаты бенчмарков

Расчёт питательности:

Сценарий	Среднее время	Пропускная способность
8 ингредиентов	`0.000385 ms`	`2.60 млн вызовов/с`
24 ингредиента	`0.001108 ms`	`0.90 млн вызовов/с`
64 ингредиента	`0.002912 ms`	`0.34 млн вызовов/с`

Оптимизатор:

Сценарий	Режим	Среднее время	Финальный статус
`dairy_low_fiber`	`minimize_cost`	`0.214929 ms`	`Infeasible`
`dairy_low_fiber`	`balance`	`0.978980 ms`	`Infeasible`
`dairy_low_fiber`	`fixed`	`1.016516 ms`	`Infeasible`

`grower_pig_lysine_deficit`	`minimize_cost`	`0.177419 ms`	`Infeasible`
`grower_pig_lysine_deficit`	`balance`	`0.688114 ms`	`Infeasible`
`grower_pig_lysine_deficit`	`fixed`	`0.715599 ms`	`Infeasible`
`layer_low_calcium`	`minimize_cost`	`0.075507 ms`	`Optimal`
`layer_low_calcium`	`balance`	`0.477134 ms`	`Optimal`
`layer_low_calcium`	`fixed`	`0.469067 ms`	`Optimal`

Вспомогательные этапы:

Операция	Сценарий	Среднее время
`auto_populate_preview`	`starter_dairy`	`50.688338 ms`
`auto_populate_preview`	`starter_swine`	`31.849788 ms`
`auto_populate_preview`	`starter_layer`	`32.868358 ms`
`screen`	`dairy_low_fiber`	`24.946853 ms`
`screen`	`grower_pig_lysine_deficit`	`24.694417 ms`
`screen`	`layer_low_calcium`	`32.404530 ms`

ИИ-агент:

Модель	Сценарий	Контекст	Задержка	Итог
`qwen3.5:4b`	`dairy_low_fiber`	`8192`	`84.23 c`	подробный ответ
`qwen3.5:4b`	`grower_pig_lysine_deficit`	`8192`	`16.75 c`	ответ получен, но только короткое вступление
`qwen3.5:4b`	`layer_low_calcium`	`8192`	`54.19 c`	подробный ответ
`qwen3.5:9b`	`dairy_low_fiber`	`4096`	`86.26 c`	подробный ответ
`qwen3.5:9b`	`grower_pig_lysine_deficit`	`4096`	`78.57 c`	подробный ответ
`qwen3.5:9b`	`layer_low_calcium`	`4096`	`74.71 c`	подробный ответ

Ключевой вывод из этих результатов: переработанный оптимизатор практически не ограничивается скоростью решателя; в трудных сценариях узким местом остаётся выполнимость набора кормов, а не вычислительная производительность.

13. Ограничения

К ограничениям текущей версии относятся:

- неполнота рыночных цен по части специализированных кормов;
- зависимость качества оптимизации от полноты feed-профилей;
- два бенчмарочных сценария остаются действительно невыполнимыми при фиксированном наборе кормов; теперь система сообщает об этом явно, а не маскирует проблему как отсутствие изменений;
- поля `max_inclusion_*` присутствуют в модели базы данных, но пока не учитываются в оптимизаторе;
- в модели валидации уже есть `DCadOutOfRange`, однако функциональный расчёт DCAD ещё не реализован;
- пресеты по птице для высокопродуктивных стад требуют ручной проверки, потому что часть нормативной базы старше современных коммерческих рекомендаций;
- невозможность заменить профессиональную биологическую интерпретацию чисто вычислительным

решением;
- вариативность поведения локальной LLM в зависимости от модели и доступных ресурсов; 4B работает быстрее, но может недоотвечать на аналитические запросы.

14. Перспективы развития

Наиболее важные направления развития:

- расширение региональной ценовой аналитики;
- учёт сезонных колебаний стоимости;
- внедрение ограничений `max_inclusion_*` и других практических лимитов включения прямо в оптимизатор;
- реализация DCAD и других проверок транзитного периода;
- обновление молочных и птичьих нормативных пресетов там, где появились более новые источники;
- автоматическая оценка качества промптов встроенного агента в дополнение к транспортным бенчмаркам;
- автоматическая генерация стандартизированных отчётов для хозяйств и научных испытаний.

15. Заключение

Felex v1.0.1 реализует связанный вычислительный контур `данные о кормах -> расчёт питательности -> стартовый рацион -> скрининг -> многоуровневая оптимизация -> стоимостной проход -> валидация -> документирование -> локальная AI-интерпретация`. Переработанный оптимизатор устраняет прежнее поведение "корма не двигаются", явно показывает невыполнимые наборы кормов и работает в связке со встроенным агентом, который сохраняет thinking mode и при этом умеет восстанавливаться после локальных сбоев Ollama. Система остаётся пригодной как для практической работы в хозяйстве, так и для технического и научного описания цифрового балансирования рационов.